

## Topic 3: How Neural Networks Work

### 3.1 The Neuron -- Basic Building Block

- A neural network is loosely inspired by the brain -- it consists of layers of artificial neurons
- Each neuron: takes multiple numerical inputs, multiplies each by a learned weight, sums them, applies an activation function
- Weights determine how important each input is -- learning means finding the right set of weights
- Activation functions (ReLU, sigmoid, softmax) add non-linearity, allowing the network to learn complex patterns
- Without activation functions, stacking layers is mathematically equivalent to a single linear transformation
- Interactive demo: <https://playground.tensorflow.org>

Single neuron computation:

```
output = activation( w1*x1 + w2*x2 + w3*x3 + bias )
```

Where:

x1, x2, x3 = input values

w1, w2, w3 = learned weights (updated during training)

bias = a learned offset value

activation = ReLU:  $\max(0, x)$  (zero for negatives, x for positives)

### 3.2 Layers and the Forward Pass

- Input layer: receives raw data (pixel values, token embeddings, numbers)
- Hidden layers: transform data through learned weights; depth enables increasingly abstract feature extraction
- Output layer: produces the final prediction (class probabilities, a generated token, a number)
- Forward pass: data flows from input -> hidden layers -> output, producing a prediction
- 'Deep' in deep learning refers to having many hidden layers (often dozens to hundreds in modern models)

### 3.3 Training -- How Networks Learn

- Loss function: measures how wrong the network's prediction is (e.g., cross-entropy for classification)
- Backpropagation: computes the gradient (direction of steepest error increase) for every weight using the chain rule

- Gradient descent: nudge every weight slightly in the direction that reduces the loss
- Learning rate: controls the size of each nudge -- too large causes instability; too small is very slow
- Epoch: one complete pass through the training dataset; training typically runs for many epochs
- The network improves incrementally with each update, gradually fitting the patterns in the data

### 3.4 Overfitting and Generalisation

- Overfitting: the model memorises training examples but performs poorly on new, unseen data
  - Underfitting: the model is too simple to capture the patterns; high error on both training and test data
  - Goal: generalisation -- good performance on data the model has never seen
  - Common solutions: more training data, dropout (randomly disable neurons during training), weight regularisation, early stopping
  - Train / validation / test split: keep separate data to detect overfitting and measure final performance honestly
-